

Polymorphism in Python

Track: Data Science | Level 1 - Day 15 | Tools: Python 3, Jupyter Notebook

Level 1 - Day 15: Polymorphism in Python

Track: Data Science

Task: Task 15 - Polymorphism and Interface Flexibility

Objective: Learn method polymorphism, duck typing, operator overloading, and creating uniform model/transformer interfaces that execute interchangeable data logic.

1. Quick Reference & Core Concepts

Polymorphism Form	Mechanism	Data Science Application
Method Overriding	Same method name across subclasses	Different loss functions exposing .compute()
Duck Typing	"If it walks like a duck..."	Any transformer implementing .transform() can be piped
Operator Overloading	__add__, __eq__, __len__	Custom metric tensor accumulation and equality checks
Built-in Polymorphism	len(), sum(), sorted()	Functions seamlessly operating on lists, tuples, dicts

2. Practical Code Examples

In [code]

```
# Example 1: Polymorphic Estimator Classes
class LinearModel:
    def predict(self, x):
        return [2.5 * val + 1.0 for val in x]

class TreeModel:
    def predict(self, x):
        return [3.0 if val >= 5 else 1.5 for val in x]

def run_predictions(estimator, input_data):
    return estimator.predict(input_data)

inputs = [2, 4, 6, 8]
print(f"Ex 1 - Linear Predictions: {run_predictions(LinearModel(), inputs)}")
print(f"Ex 1 - Tree Predictions: {run_predictions(TreeModel(), inputs)}")
```

In [code]

```
# Example 2: Duck Typing with Custom Data Preprocessors
class StandardScaler:
    def transform(self, data):
        mean = sum(data) / len(data)
        std = (sum((x - mean) ** 2 for x in data) / len(data)) ** 0.5
        return [(x - mean) / std for x in data]

class Binarizer:
    def transform(self, data):
        return [1 if x >= 50 else 0 for x in data]

dataset = [20, 50, 80]
for transformer in [StandardScaler(), Binarizer()]:
    print(f"Ex 2 - Transformed via {transformer.__class__.__name__}: {[round(x, 2) for x in
transformer.transform(dataset)]}")
```

3. Applied Data Science Pipeline

Applied Data Pipeline: Polymorphic Metric Evaluator Engine

In [code]

```
class AccuracyMetric:
    def calculate(self, y_true, y_pred):
        correct = sum(1 for a, b in zip(y_true, y_pred) if a == b)
        return correct / len(y_true)

class PrecisionMetric:
    def calculate(self, y_true, y_pred):
        tp = sum(1 for a, b in zip(y_true, y_pred) if a == 1 and b == 1)
        fp = sum(1 for a, b in zip(y_true, y_pred) if a == 0 and b == 1)
        return tp / (tp + fp) if (tp + fp) > 0 else 0.0

y_actual = [1, 0, 1, 1, 0, 1]
y_forecast = [1, 0, 0, 1, 0, 1]

metrics = [AccuracyMetric(), PrecisionMetric()]
for m in metrics:
    score = m.calculate(y_actual, y_forecast)
    print(f"{m.__class__.__name__}: {score:.2%}")
```